HPC/hpc-2.cpp

```cpp
/*
    Mustansir Rangwala
    3083
    B2
    Problem Statement:
    Write a program to implement Parallel Bubble Sort and Merge sort using OpenMP. Use existing algorithms and measure
    the performance of sequential and parallel algorithms.
*/


#include <iostream>
#include <omp.h>
#include <random>
#include <vector>
#include <climits>
#include <chrono>
#include <thread>
#include <string>
using namespace std;

/**
 * Functions in use.
 */

vector<int> getRandomVector(int size);
void executeAndCalculateElapsedTime(vector<int> nums, void (*sortMethod)(vector<int>));


void normalBubbleSort(vector<int> nums);
void parallelBubbleSort(vector<int> nums);

void merge(vector<int>& nums, int left, int mid, int right);
void mergeSortHelper(vector<int>& nums, int left, int right);
void normalMergeSort(vector<int> nums);
void parallelMergeSort(vector<int> nums);

void printVector(vector<int>& nums);


int main(int argc, char* argv[]){
    int size;
    try {
        if (argc != 3) throw invalid_argument("Invalid argument count");

        string flag = argv[1];

        if (flag != "-y" && flag != "-n") throw invalid_argument("Invalid flag");

        size = stoi(argv[2]);

```

```
        if (size ≤ 0) throw invalid_argument("Size must be positive");

        cout << "Flag: " << flag << endl;
        cout << "Size: " << size << endl;

    } catch ( ... ) {
        cerr << "USAGE: ./out <show_vector_flag> <number_of_elements>" << endl;
        return 1;
    }



    vector<int> nums = getRandomVector(size);


    executeAndCalculateElapsedTime(nums, normalBubbleSort);
    executeAndCalculateElapsedTime(nums, parallelBubbleSort);

    executeAndCalculateElapsedTime(nums, normalMergeSort);
    executeAndCalculateElapsedTime(nums, parallelMergeSort);


    return 0;

}



vector<int> getRandomVector(int size) {
    vector<int> nums;
    random_device rd;
    mt19937 gen(rd());
    uniform_int_distribution<int> dist(INT_MIN, INT_MAX);
    while(size--) nums.push_back(dist(gen));
    return nums;
}

void printVector(vector<int>& nums) {
    for(const int& num : nums) cout << num << " ";
    cout << "\n";
}

void executeAndCalculateElapsedTime(vector<int> nums, void (*sortMethod)(vector<int>)) {
    using Clock = std::chrono::steady_clock;
    using std::chrono::duration_cast;
    using std::chrono::milliseconds;

    // Capture the start time
    Clock::time_point start = Clock::now();

    sortMethod(nums);

    // Capture the end time
```

```
    Clock::time_point end = Clock::now();

    // Calculate the difference and cast to desired units
    milliseconds diff = duration_cast<milliseconds>(end - start);

    std::cout << "Operation took " << diff.count() << "ms" << std::endl;
}

void normalBubbleSort(vector<int> nums){
    bool sorted = false;
    while(!sorted){
        sorted = true;
        for(int i=0;i<nums.size()-1;i++){
            if(nums[i] > nums[i+1]) {
                swap(nums[i], nums[i+1]);
                sorted = false;
            }
        }
    }

}

/**
// Source - https://stackoverflow.com/a/2028227
// Posted by Nick Dandoulakis
// Retrieved 2026-03-02, License - CC BY-SA 2.5

1.  For k = 0 to n-2
2.  If k is even then
3.      for i = 0 to (n/2)-1 do in parallel
4.          If A[2i] > A[2i+1] then
5.              Exchange A[2i] ↔ A[2i+1]
6.  Else
7.      for i = 0 to (n/2)-2 do in parallel
8.          If A[2i+1] > A[2i+2] then
9.              Exchange A[2i+1] ↔ A[2i+2]
10. Next k

 */

void parallelBubbleSort(vector<int> nums) {
    int n = nums.size();

    for(int k = 0; k < n; k++) {
        if(k % 2 == 0) {
            // Even phase: compare (0,1), (2,3), (4,5)...
            #pragma omp parallel for
            for(int i = 0; i < n/2; i++) {
                if(nums[2*i] > nums[2*i + 1]) {
                    swap(nums[2*i], nums[2*i + 1]);
                }
            }
        } else {
            // Odd phase: compare (1,2), (3,4), (5,6)...
```

```
            #pragma omp parallel for
            for(int i = 0; i < (n-1)/2; i++) {
                if(nums[2*i + 1] > nums[2*i + 2]) {
                    swap(nums[2*i + 1], nums[2*i + 2]);
                }
            }
        }
    }
}


void merge(vector<int>& nums, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    vector<int> L(n1), R(n2);

    for(int i = 0; i < n1; i++)
        L[i] = nums[left + i];
    for(int j = 0; j < n2; j++)
        R[j] = nums[mid + 1 + j];

    int i = 0, j = 0, k = left;

    while(i < n1 && j < n2) {
        if(L[i] ≤ R[j])
            nums[k++] = L[i++];
        else
            nums[k++] = R[j++];
    }

    while(i < n1)
        nums[k++] = L[i++];

    while(j < n2)
        nums[k++] = R[j++];
}

void mergeSortHelper(vector<int>& nums, int left, int right) {
    if(left ≥ right) return;

    int mid = left + (right - left) / 2;
    mergeSortHelper(nums, left, mid);
    mergeSortHelper(nums, mid + 1, right);
    merge(nums, left, mid, right);
}

void normalMergeSort(vector<int> nums) {
    if(nums.empty()) return;
    mergeSortHelper(nums, 0, nums.size() - 1);
}

void parallelMergeSortHelper(vector<int>& nums, int left, int right, int depth) {
    if(left ≥ right) return;
```

```

    int mid = left + (right - left) / 2;

    if(depth <= 0) {
        parallelMergeSortHelper(nums, left, mid, depth - 1);
        parallelMergeSortHelper(nums, mid + 1, right, depth - 1);
    } else {
        #pragma omp parallel sections
        {
            #pragma omp section
            parallelMergeSortHelper(nums, left, mid, depth - 1);

            #pragma omp section
            parallelMergeSortHelper(nums, mid + 1, right, depth - 1);
        }
    }

    merge(nums, left, mid, right);
}

void parallelMergeSort(vector<int> nums) {
    if(nums.empty()) return;
    int maxDepth = 32;
    parallelMergeSortHelper(nums, 0, nums.size() - 1, maxDepth);
}


/**
 * OUTPUT:
 * PS D:\8th SEM\HPC\HPC_CODES> g++ hpc2.cpp -o out
 * PS D:\8th SEM\HPC\HPC_CODES> ./out -y 10000
 * Flag: -y
 * Size: 10000
 * Operation took 1144ms
 * Operation took 587ms
 * Operation took 8ms
 * Operation took 9ms
 */
```